

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC NGOẠI NGỮ - TIN HỌC THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



MÔN HỌC : DỊCH NGƯỢC

**CÁC KỸ THUẬT DỊCH NGƯỢC VÀ ỨNG DỤNG VÀO INTERNET
DOWNLOAD MANAGER**

Giáo Viên Hướng Dẫn : ThS. Phạm Đình Thắng

Sinh viên thực hiện :

1. Bùi Viết Nguyên Chương – MSSV:
23DH110383
2. Phan Trần Anh Đức – MSSV: 23DH110838

Tp. Hồ Chí Minh, tháng 04 năm 2026

LỜI NÓI ĐẦU

Trong bối cảnh công nghệ thông tin phát triển mạnh mẽ, các phần mềm ngày càng trở nên phức tạp và đóng vai trò quan trọng trong nhiều lĩnh vực của đời sống. Tuy nhiên, không phải lúc nào mã nguồn của các phần mềm này cũng được công khai. Điều này đặt ra nhu cầu cần phải hiểu và phân tích cách thức hoạt động của chương trình ở mức thấp hơn, từ đó hình thành nên lĩnh vực dịch ngược (reverse engineering).

Dịch ngược là một kỹ thuật quan trọng trong an toàn thông tin và phát triển phần mềm, cho phép người phân tích khám phá cấu trúc, logic và cơ chế hoạt động của một chương trình thông qua mã máy. Thông qua các phương pháp như phân tích tĩnh, phân tích động và debugging, người nghiên cứu có thể hiểu rõ luồng thực thi, các điều kiện rẽ nhánh cũng như cơ chế kiểm soát bên trong phần mềm.

Trong khuôn khổ môn học này, nhóm tiến hành nghiên cứu các kỹ thuật dịch ngược cơ bản và áp dụng vào phân tích một phần mềm thực tế là WinRAR. Thông qua quá trình thực nghiệm, nhóm không chỉ củng cố kiến thức lý thuyết mà còn rèn luyện kỹ năng phân tích chương trình, từ đó hiểu rõ hơn về cách một phần mềm thương mại vận hành ở mức độ chi tiết.

LỜI CẢM ƠN

Kính chào thầy Phạm Đình Thắng.

Hôm nay, em vô cùng vinh dự được đại diện nhóm để trình bày bài báo cáo về chủ đề Dịch Ngược trong khuôn khổ môn học của thầy. Đây là một đề tài rất thú vị và bổ ích, giúp em không chỉ nâng cao kiến thức chuyên môn mà còn rèn luyện được các kỹ năng phân tích, tư duy phản biện và thực hành điều tra mạng.

Trước hết, em xin gửi lời cảm ơn sâu sắc đến thầy Phạm Đình Thắng, người đã tận tâm giảng dạy, chỉ dẫn từng bước và chia sẻ những kinh nghiệm thực tế quý báu. Nhờ sự hỗ trợ và hướng dẫn nhiệt tình của thầy, em đã có cơ hội hiểu rõ hơn về các khía cạnh phức tạp của lĩnh vực này, từ các nguyên tắc cơ bản cho đến những công cụ và kỹ thuật tiên tiến trong điều tra mạng.

Bên cạnh đó, em cũng xin bày tỏ lòng biết ơn đến các bạn sinh viên trong lớp, những người đã đồng hành, thảo luận và góp ý cho em trong quá trình nghiên cứu và chuẩn bị bài báo cáo này. Nhờ những ý kiến đóng góp chân thành của các bạn, em đã có thêm nhiều góc nhìn mới mẻ, giúp bài báo cáo của mình trở nên hoàn thiện hơn.

Ngoài ra, em cũng muốn cảm ơn Ban Giám hiệu nhà trường, khoa Công nghệ thông tin và đặc biệt là sự hỗ trợ từ nhà trường trong việc tạo điều kiện học tập, nghiên cứu và ứng dụng thực tế. Những cơ sở vật chất và nguồn tài liệu phong phú của nhà trường đã giúp em rất nhiều trong việc phát triển bài báo cáo.

Không thể không kể đến sự động viên và khích lệ từ gia đình và bạn bè. Họ luôn là nguồn động lực lớn, giúp em vượt qua những khó khăn trong học tập và nghiên cứu.

Em nhận thức rõ rằng, dù đã nỗ lực hết mình, bài báo cáo này vẫn khó tránh khỏi những hạn chế và thiếu sót. Do đó, em rất mong nhận được những góp ý chân thành từ thầy và các bạn sinh viên để em có thể cải thiện và hoàn thiện hơn trong tương lai.

Cuối cùng, em xin kính chúc thầy Phạm Đình Thắng luôn dồi dào sức khỏe, hạnh phúc và thành công trong sự nghiệp giảng dạy. Chúc các bạn sinh viên trong lớp học tập tiến bộ, đạt được nhiều thành công trên con đường chinh phục tri thức.

Em xin chân thành cảm ơn!

NHẬN XÉT CỦA GIẢNG VIÊN

Bảng chữ ký

Tác giả:

Tên: _____

Chữ ký: _____

Vị trí: _____

Ngày: _____

Tên:

Chữ ký: _____

Vị trí: _____

Ngày: _____

Tên:

Chữ ký: _____

Vị trí: _____

Ngày: _____

Người điều chỉnh:

Tên: _____

Chữ ký: _____

Vị trí: _____

Ngày: _____

Người duyệt:

Tên: _____

Chữ ký: _____

Vị trí: _____

Ngày: _____

Mục lục

LỜI NÓI ĐẦU	2
LỜI CẢM ƠN.....	3
NHẬN XÉT CỦA GIẢNG VIÊN	4
CHƯƠNG I: CƠ SỞ LÝ THUYẾT	8
1. Giới thiệu Dịch Ngược.....	8
1.1 Dịch ngược là gì?.....	8
1.2 Sự cần thiết của Dịch Ngược.....	8
2. Các kỹ thuật tấn công và vai trò của dịch ngược	8
2.1. Vai trò của dịch ngược trong phân tích phần mềm	9
2.2. Kỹ thuật obfuscation và packing.....	9
2.3. Phân tích tĩnh và động.....	10
3. Các kỹ thuật chính trong dịch ngược	10
3.1. Phân tích tĩnh (Static Analysis)	11
3.1.1. Khái niệm	11
3.1.2. Dữ liệu thu được.....	11
3.1.3. Vai trò.....	11
3.1.4. Các loại kỹ thuật.....	11
3.1.5. Công cụ tiêu biểu	11
3.2. Phân tích động (Dynamic Analysis)	11
3.2.1. Khái niệm	11
3.2.2. Dữ liệu thu được.....	11
3.2.3. Vai trò.....	12
3.2.4. Các loại kỹ thuật.....	12
3.2.5. Công cụ.....	12
3.3. Debugging và phân tích luồng thực thi	12
3.3.1. Khái niệm	12
3.3.2. Dữ liệu thu được.....	12
3.3.3. Vai trò.....	12
3.3.4. Các loại kỹ thuật.....	13
3.3.5. Công cụ.....	13

4. Quy trình ứng dụng dịch ngược vào phân tích IDMan	13
4.1. Khảo sát chương trình và chuẩn bị môi trường.....	13
4.2. Phân tích cấu trúc và các chức năng chính	13
4.3. Theo dõi hành vi khi thực thi.....	14
4.4. Tổng hợp cơ chế hoạt động và rút ra kết luận	14
CHƯƠNG II: THỰC NGHIỆM.....	15
1. Cách 1: Bybass bằng x32dbg để ép thông báo Licensed.....	15
1.1. Giới thiệu IDMan:.....	15
1.2. Môi trường & công cụ sử dụng.....	15
1.3. Quy trình thực hiện + kết quả phân tích	15
1.3.1. Thu thập thông tin	15
1.3.2. Truy vết ngược luồng thực thi	16
1.3.5. Kiểm tra kết quả thực nghiệm	19
2. Cách 2: Bypass bằng x32dbg để bỏ thông báo check Serial	20
2.1. Thu thập thông tin và cơ sở lý thuyết.....	20
2.2. Truy vết quá trình phần mềm tìm kiếm hàm check serial.....	20
2.3. Kết quả cuối.....	23
CHƯƠNG III: KẾT QUẢ THỰC NGHIỆM.....	25
1. Tổng kết kết quả đạt được	25
2. Ý nghĩa thực tiễn.....	25
3. Hạn chế và hướng phát triển	25
Tài liệu tham khảo:.....	27

CHƯƠNG I: CƠ SỞ LÝ THUYẾT

1. Giới thiệu Dịch Ngược

1.1 Dịch ngược là gì?

Dịch ngược (Reverse Engineering) Là quá trình phân tích một chương trình, ứng dụng, hoặc mã nguồn để hiểu cách nó hoạt động, thường là không có sẵn mã nguồn gốc. Mục đích là làm sáng tỏ cấu trúc, chức năng, và logic bên trong của phần mềm.

1.2 Sự cần thiết của Dịch Ngược

Thứ nhất: Phân tích mối đe dọa: Kỹ thuật dịch ngược được sử dụng để phân tích phần mềm độc hại (malware) và các mối đe dọa khác nhằm xác định khả năng, mục tiêu, và cách thức lây lan của chúng. Việc này giúp các nhà nghiên cứu bảo mật hiểu được cách thức hoạt động của phần mềm độc hại, từ đó xây dựng các công cụ và biện pháp phòng vệ hiệu quả hơn.

Thứ hai: Tìm lỗ hổng: Nó giúp các nhà nghiên cứu bảo mật và chuyên gia kiểm thử xâm nhập (penetration testers) tìm kiếm các lỗ hổng bảo mật trong phần mềm.

Thứ ba, Tương tác và khả năng tương thích: Kỹ thuật dịch ngược có thể được sử dụng để tạo ra các giao diện tương thích hoặc cho phép hệ thống làm việc với các hệ thống khác khi không có tài liệu kỹ thuật đầy đủ..

Ngoài ra, trong khuôn khổ pháp lý, kỹ thuật dịch ngược có thể được sử dụng để kiểm tra, đánh giá, hoặc phát triển các sản phẩm tương thích

2. Các kỹ thuật tấn công và vai trò của dịch ngược

Trong lĩnh vực an toàn thông tin, các cuộc tấn công ngày càng trở nên tinh vi và khó phát hiện. Đặc biệt, phần lớn các cuộc tấn công hiện nay đều sử dụng phần mềm độc hại (malware) hoặc các kỹ thuật che giấu nhằm tránh bị phát hiện.

Dịch ngược đóng vai trò quan trọng trong việc phân tích các chương trình này, giúp hiểu rõ cách thức hoạt động, mục tiêu và hành vi của chúng, từ đó hỗ trợ việc phát hiện và phòng chống hiệu quả.

2.1. Vai trò của dịch ngược trong phân tích phần mềm

Dịch ngược (reverse engineering) đóng vai trò quan trọng trong việc phân tích và hiểu hoạt động của các chương trình khi không có mã nguồn. Thông qua việc chuyển đổi mã máy sang dạng dễ đọc hơn như assembly hoặc pseudocode, người phân tích có thể nắm được cấu trúc, luồng thực thi và chức năng của phần mềm.

Trong thực tế, dịch ngược được sử dụng để:

- Phân tích logic hoạt động của chương trình
- Xác định các điều kiện rẽ nhánh và luồng điều khiển
- Theo dõi sự thay đổi của thanh ghi (register) và cờ trạng thái (flags)
- Hiểu cơ chế kiểm tra, bảo vệ hoặc xác thực trong phần mềm
- Hỗ trợ việc chỉnh sửa, vá lỗi hoặc can thiệp vào hành vi chương trình

Đặc biệt, trong các phần mềm thương mại như WinRAR, dịch ngược cho phép người phân tích xác định cách chương trình kiểm tra trạng thái bản quyền, từ đó hiểu được cơ chế hoạt động nội tại của hệ thống mà không cần truy cập vào mã nguồn gốc.

2.2. Kỹ thuật obfuscation và packing

Để tránh bị phân tích, malware thường sử dụng các kỹ thuật như:

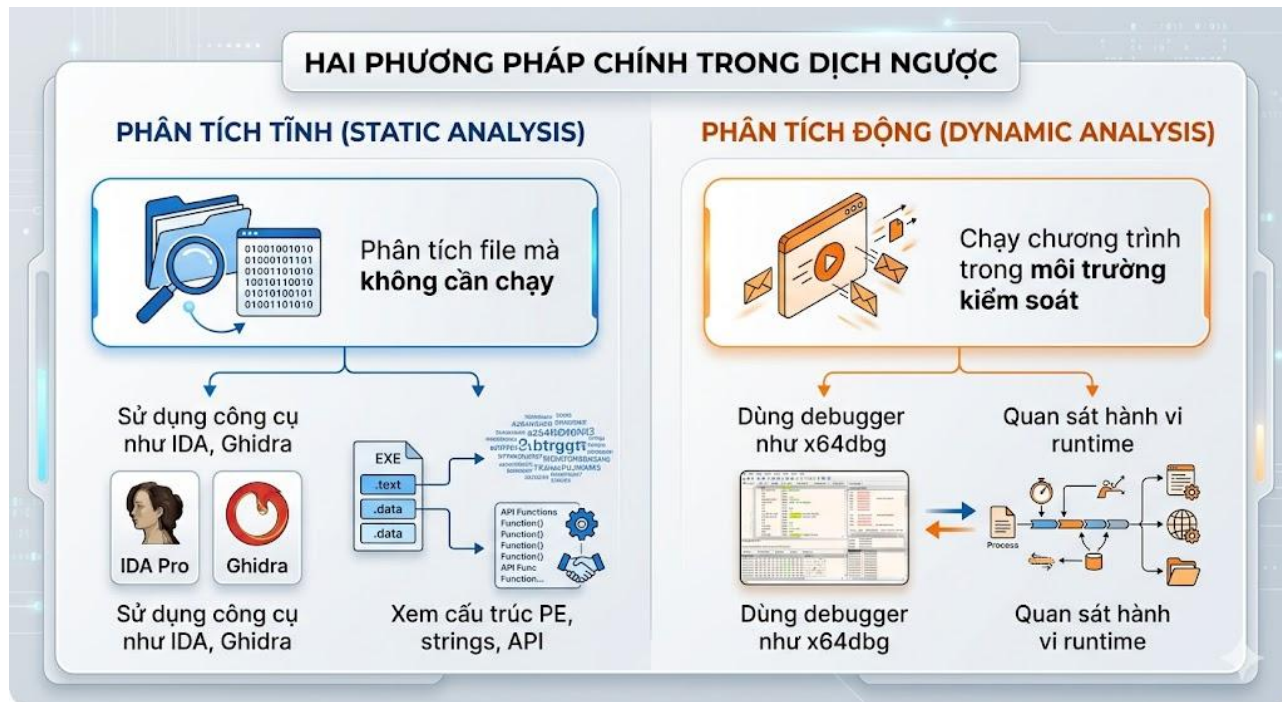
- **Obfuscation (làm rối mã):** biến đổi code để khó đọc
- **Packing (đóng gói):** nén hoặc mã hóa file thực thi

Các kỹ thuật này khiến việc phân tích trở nên khó khăn hơn, vì code thực sự chỉ được giải mã khi chương trình chạy.

Dịch ngược giúp:

- Unpack chương trình
- Khôi phục mã gốc
- Phân tích logic thực sự bên trong

2.3. Phân tích tĩnh và động



Phân tích tĩnh (Static Analysis)

- Phân tích file mà không cần nạp lên RAM
- Sử dụng công cụ như IDA, Ghidra
- Xem cấu trúc PE, strings, API

Phân tích động (Dynamic Analysis)

- Chạy chương trình trong môi trường kiểm soát
- Dùng debugger như x64dbg
- Quan sát hành vi tiến trình và các thanh ghi, stack.

3. Các kỹ thuật chính trong dịch ngược

Dịch ngược là quá trình phân tích chương trình ở mức mã máy nhằm hiểu rõ cấu trúc và hành vi của phần mềm. Để thực hiện hiệu quả, cần áp dụng nhiều kỹ thuật khác nhau, trong đó phổ biến nhất là phân tích tĩnh, phân tích động và debugging.

3.1. Phân tích tĩnh (Static Analysis)

3.1.1. Khái niệm

Phân tích tĩnh là quá trình nghiên cứu file thực thi mà không cần chạy chương trình. Kỹ thuật này giúp hiểu cấu trúc tổng thể và các thành phần bên trong của phần mềm.

3.1.2. Dữ liệu thu được

- Strings (chuỗi ký tự)
- Import/Export functions
- Cấu trúc PE (Portable Executable)
- API calls

3.1.3. Vai trò

- Xác định hành vi đáng ngờ ban đầu
- Tìm thông tin như domain, file path
- Hiểu sơ bộ logic chương trình

3.1.4. Các loại kỹ thuật

- Disassembly
- Decompile/Pseudocode.
- String analysis

3.1.5. Công cụ tiêu biểu

- IDA Pro
- Ghidra
- PEiD

3.2. Phân tích động (Dynamic Analysis)

3.2.1. Khái niệm

Phân tích động là việc chạy chương trình trong môi trường kiểm soát để quan sát hành vi thực tế của nó.

3.2.2. Dữ liệu thu được

- Hành vi
- Kết nối mạng

- File tạo ra
- Process được spawn

3.2.3. Vai trò

- Phát hiện hành vi ẩn mà static không thấy
- Quan sát malware hoạt động thực tế
- Phát hiện C2 communication

3.2.4. Các loại kỹ thuật

- Sandbox analysis
- Monitoring process
- API tracing

3.2.5. Công cụ

- Process Monitor
- Wireshark
- Cuckoo Sandbox
- Any.run

3.3. Debugging và phân tích luồng thực thi

3.3.1. Khái niệm

Debugging là kỹ thuật theo dõi chương trình từng bước khi đang chạy nhằm hiểu chi tiết luồng thực thi.

3.3.2. Dữ liệu thu được

- Giá trị thanh ghi (register)
- Bộ nhớ (memory)
- Stack và call stack
- Flags

3.3.3. Vai trò

- Xác định chính xác logic chương trình
- Phát hiện đoạn code quan trọng

- Bypass cơ chế bảo vệ

3.3.4. Các loại kỹ thuật

- Breakpoint
- Step into / step over
- Call Stack

3.3.5. Công cụ

- x64dbg
- OllyDbg
- WinDbg

-> Trong bài thực nghiệm, kỹ thuật phân tích động và debugging được áp dụng để xác định hành vi của file thực thi

4. Quy trình ứng dụng dịch ngược vào phân tích IDMan

Đối với một phần mềm phổ biến như IDMan, dịch ngược có thể được áp dụng để tìm hiểu cấu trúc chương trình, cơ chế thực thi các chức năng chính và cách phần mềm tương tác với hệ điều hành.

Mục tiêu của quá trình này không phải phá hoại vào phần mềm, mà là phân tích để hiểu rõ thiết kế và nguyên lý hoạt động của ứng dụng ở mức thấp.

4.1. Khảo sát chương trình và chuẩn bị môi trường

Trước hết, cần xác định file thực thi chính của IDMan, phiên bản phân tích và môi trường chạy thử phù hợp. Người phân tích tiến hành thu thập các thông tin cơ bản về tệp tin như cấu trúc PE (Portable Executable), kiểm tra các lớp bảo vệ (Packer), các thư viện liên kết động (DLL) và bảng chuỗi ký tự (String Table). Việc chuẩn bị một môi trường sạch với đầy đủ công cụ giúp quá trình phân tích diễn ra nhất quán và tránh các tác động không mong muốn đến hệ thống thực.

4.2. Phân tích cấu trúc và các chức năng chính

Ở giai đoạn này, các công cụ dịch ngược (Disassembler/Decompiler) như IDA Pro được sử dụng để bóc tách các module quan trọng của IDMan như: cơ chế bắt link (Integration), thuật toán chia nhỏ tệp tin (Segmentation), và hệ thống quản lý hàng đợi. Mục tiêu trọng tâm là nhận diện cách phần mềm tổ chức mã nguồn, các điểm vào (Entry Point) của các hàm xử lý logic và mối liên hệ giữa giao diện

người dùng với các dịch vụ chạy ngầm. Qua đó, người phân tích nắm được sơ đồ tổng thể của một trình quản lý tải xuống phức tạp.

4.3. Theo dõi hành vi khi thực thi

Sau khi phân tích tĩnh, IDMan được vận hành trong môi trường kiểm soát để quan sát hành vi thực tế (Dynamic Analysis). Người phân tích sử dụng Debugger và các công cụ giám sát hệ thống để theo dõi luồng thực thi, các lời gọi API mạng, thao tác đọc/ghi Registry để lưu trữ cấu hình, và quá trình ghi tệp tạm trên đĩa cứng. Việc theo dõi này giúp xác thực các giả thuyết từ mã nguồn, đặc biệt là cách chương trình phản hồi khi xác thực thông tin bản quyền hoặc khi tương tác với các trình duyệt web.

4.4. Tổng hợp cơ chế hoạt động và rút ra kết luận

Kết quả phân tích được hệ thống hóa để làm rõ cơ chế vận hành cốt lõi của IDMan, từ cách thức tối ưu tốc độ tải xuống đến phương pháp bảo vệ phần mềm. Báo cáo tổng hợp lại các kỹ thuật dịch ngược đã áp dụng và hiệu quả của chúng trong việc làm sáng tỏ các thành phần bị đóng gói hoặc mã hóa. Thông qua đó, người học không chỉ hiểu sâu về kiến trúc của IDMan mà còn xây dựng được tư duy phân tích hệ thống, phục vụ cho việc nghiên cứu an toàn thông tin và tối ưu hóa phần mềm trong tương lai.

- ☐ **Quy trình này không phải tuyến tính tuyệt đối — nó là chu trình lặp lại và mỗi bước có thể tương tác với các bước khác**

CHƯƠNG II: THỰC NGHIỆM

1. Cách 1: Bybass bằng x32dbg để ép thông báo Licensed

1.1. Giới thiệu IDMan:

Internet Download Manager (IDMan) là một công cụ hỗ trợ quản lý và tăng tốc độ tải xuống dữ liệu phổ biến trên hệ điều hành Windows, được phát triển bởi Tonec Inc. Phần mềm này nổi bật với khả năng phân đoạn tệp tin thông minh và kỹ thuật tải xuống đa luồng an toàn, giúp tối ưu hóa băng thông và đẩy nhanh tốc độ tải lên gấp nhiều lần so với các trình duyệt thông thường.

Ngoài ra, IDMan còn cung cấp các tính năng mạnh mẽ như tự động bắt liên kết (Link grabbing) từ trình duyệt, phục hồi các bản tải xuống bị gián đoạn do lỗi mạng hoặc mất điện, và lập lịch trình tải tệp. Nhờ giao diện trực quan cùng khả năng tương thích cao với hầu hết các trình duyệt phổ biến hiện nay, IDMan đã trở thành giải pháp hàng đầu được người dùng cá nhân và doanh nghiệp lựa chọn để quản lý dữ liệu trực tuyến.

1.2. Môi trường & công cụ sử dụng

- Hệ điều hành sử dụng để phân tích: Window
- Công cụ phân tích: x32dbg
- File thực thi được sử dụng: IDMan.exe

1.3 Quy trình thực hiện + kết quả phân tích

1.3.1. Thu thập thông tin

Trong quá trình tìm hiểu về cơ chế kích hoạt của IDMan, nhóm nghiên cứu đã phân tích quy trình đăng ký bản quyền thông qua giao diện chính của phần mềm. Giả thuyết được đặt ra là sau khi người dùng nhập thông tin họ tên, email và số sê-ri (Serial Number), chương trình sẽ tiến hành xác thực và lưu trữ trạng thái đăng ký vào một vị trí cụ thể trong hệ thống để đối chiếu ở những lần khởi động sau.

Dựa vào giả thuyết này, nhóm sử dụng công cụ x32dbg (do IDMan là ứng dụng 32-bit) để tiến hành tra cứu các chuỗi ký tự (String References) trên

toàn bộ các module đang thực thi. Kết quả thu được đã giúp khoanh vùng các chuỗi ký tự đáng ngờ liên quan đến quá trình kiểm tra bản quyền, tiêu biểu như: "Registration", "Serial", "FName", "**Licensed**". Những thông tin này là manh mối quan trọng để xác định các hàm API mà phần mềm sử dụng để truy xuất dữ liệu đăng ký từ hệ thống.

1.3.2. Truy vết ngược luồng thực thi

Thay vì đọc mã lệnh từ điểm bắt đầu (Entry Point) xuống dưới (Top-Down) vốn chứa hàng triệu dòng lệnh phức tạp, nhóm áp dụng phương pháp truy xuất ngược từ đáy (Bottom-Up). Từ chuỗi "licensed" đã xác định ở bước trên, nhóm thực hiện đặt các break point để xem hàm nào sẽ được gọi vào, và khi thực hiện debug thì thấy chỉ có dòng dưới hit, đó là:

```
00E8A441 | 68 34AE0801          | push idman.108AE34          |
108AE34:"This product is licensed to:"
```

Khi đứng tại lệnh nạp chuỗi này, nhóm tiến hành cuộn ngược mã lệnh lên phía trên (scroll up) khoảng 20 dòng để tìm kiếm các khối lệnh rẽ nhánh có điều kiện (je, jne, jz). Đây là phương pháp tối ưu để trả lời câu hỏi: "Điều kiện logic nào đã cho phép luồng thực thi đi đến lệnh thông báo licensed này?". Bắt đầu phân tích từ đầu hàm, ta có:

Các lệnh đầu hàm 1

Trước lệnh nhảy (Dòng 00E8A438)

Lệnh: cmp dword ptr ds:[edx-C], ebx

Đây là lệnh So sánh (Compare). Chương trình đang so sánh một giá trị được lấy từ bộ nhớ (có vẻ là biến lưu trạng thái đăng ký hoặc kết quả trả về từ hàm kiểm tra Registry trước đó) với thanh ghi ebx.

Kết quả: Lệnh này sẽ thiết lập các Flags (cờ hiệu) trong CPU. Nếu hai giá trị bằng nhau, cờ Zero Flag (ZF) sẽ được dựng lên (ZF = 1).

Lệnh nhảy điều kiện (Dòng 00E8A43B - Điểm mấu chốt)

Lệnh gốc: je idman.E8A76B (JE = Jump if Equal - Nhảy nếu bằng nhau).

Logic mặc định: Nếu kết quả so sánh ở trên cho thấy phần mềm chưa đăng ký (giả sử giá trị không khớp), lệnh JE sẽ kích hoạt và điều hướng chương trình nhảy thẳng đến địa chỉ E8A76B (thường là đoạn mã bỏ qua phần Licensed hoặc hiện thông báo Trial).

Lệnh mới: jne idman.E8A76B (JNE = Jump if Not Equal - Nhảy nếu KHÔNG bằng nhau).

Kết quả: * Nếu trạng thái là "Chưa đăng ký" (lẽ ra phải nhảy để thoát), thì nay vì nó "Không bằng" nên lệnh JNE sẽ không nhảy nữa.

Chương trình bị "ép" phải thực thi tiếp xuống dòng 00E8A441.

Hệ quả: CPU sẽ nạp chuỗi "This product is licensed to:" và thực hiện các lệnh phía sau, dẫn đến việc IDMan hiển thị thông báo đã có bản quyền trong phần Registration.

00E8A434	> 8B5424 18	mov edx,dword ptr ss:[esp+18]	
00E8A438	395A F4	cmp dword ptr ds:[edx-C],ebx	
00E8A43B	0F85 2A030000	jne idman.E8A76B	
00E8A441	68 34AE0801	push idman.108AE34	108AE34:"This product is licensed to:"
00E8A446	8D4C24 2C	lea ecx,dword ptr ss:[esp+2C]	
00E8A44A	E8 918CF7FF	call <idman.sub_E030E0>	
00E8A44F	C68424 E4000000 01	mov byte ptr ss:[esp+E4],1	
00E8A457	8D4424 28	lea eax,dword ptr ss:[esp+28]	

Sau khi kiểm tra thêm ở trên nhóm có phát hiện 1 lệnh nhảy dài xuống địa chỉ E8A783, việc này khiến nếu lệnh nhảy được thực hiện thì chương trình sẽ bỏ qua đoạn thông báo đăng ký ở địa chỉ E8A441:

00E8A2DB	33DB	xor ebx,ebx	
00E8A2DD	894C24 34	mov dword ptr ss:[esp+34],ecx	
00E8A2E1	BD 01000000	mov ebp,1	
00E8A2E6	391D DC8C1501	cmp dword ptr ds:[1158CDC],ebx	
00E8A2EC	0F85 91040000	jne idman.E8A783	
00E8A2F2	8D4C24 18	lea ecx,dword ptr ss:[esp+18]	
00E8A2F6	E8 F581F7FF	call <idman.sub_E024F0>	
00E8A2FB	899C24 E4000000	mov dword ptr ss:[esp+E4],ebx	
00E8A302	8D4424 20	lea eax,dword ptr ss:[esp+20]	[esp+20]:TpCallbackIndependent+140
00E8A306	50	push eax	
00E8A307	55	push ebp	
00E8A308	53	push ebx	
00E8A309	68 68DE0701	push idman.107DE68	sub_E8A309
00E8A30E	68 02000080	push 80000002	
00E8A313	5F15 88C88701	call ds:[F-BA888701]	

Lệnh nhảy điều kiện (Dòng 00E8A2EC - Điểm rẽ nhánh)

Lệnh gốc: jne idman.E8A783 (JNE = Jump if Not Equal - Nhảy nếu KHÔNG bằng nhau).

Luồng thực thi mặc định: Nếu bài kiểm tra trên cho kết quả "Chưa đăng ký" (hai giá trị so sánh không bằng nhau), lệnh JNE sẽ kích hoạt.

Hậu quả: Chương trình sẽ thực hiện một cú nhảy dài (OF 85) đến địa chỉ E8A783. Địa chỉ này nằm rất xa, hoàn toàn thoát khỏi vùng mã nguồn chứa thông báo Licensed. Báo cáo của bạn sẽ ghi nhận đây là nhánh "Thoát" (Exit/Bailout branch).

Hệ quả trực tiếp: Nếu lệnh JNE này được thực thi, sẽ không bao giờ nhìn thấy thông báo Licensed, dù có Patch đoạn JE ở trước như thế nào đi nữa.

Bằng cách đổi lệnh nhảy, nhóm sẽ lật ngược logic bảo vệ:

Lệnh mới: je idman.E8A783 (JE = Jump if Equal - Nhảy nếu bằng nhau).

Logic mới: * Bây giờ, nếu bài kiểm tra cho kết quả "Chưa đăng ký" (lẽ ra phải nhảy thoát), thì lệnh JE mới sẽ không nhảy (vì chúng không bằng nhau).

Hệ quả: Chương trình bị "ép" phải trôi xuống dòng 00E8A2F2, tiếp tục thực thi các lệnh phía sau (trong đó có bài kiểm tra thứ hai và cuối cùng là đoạn

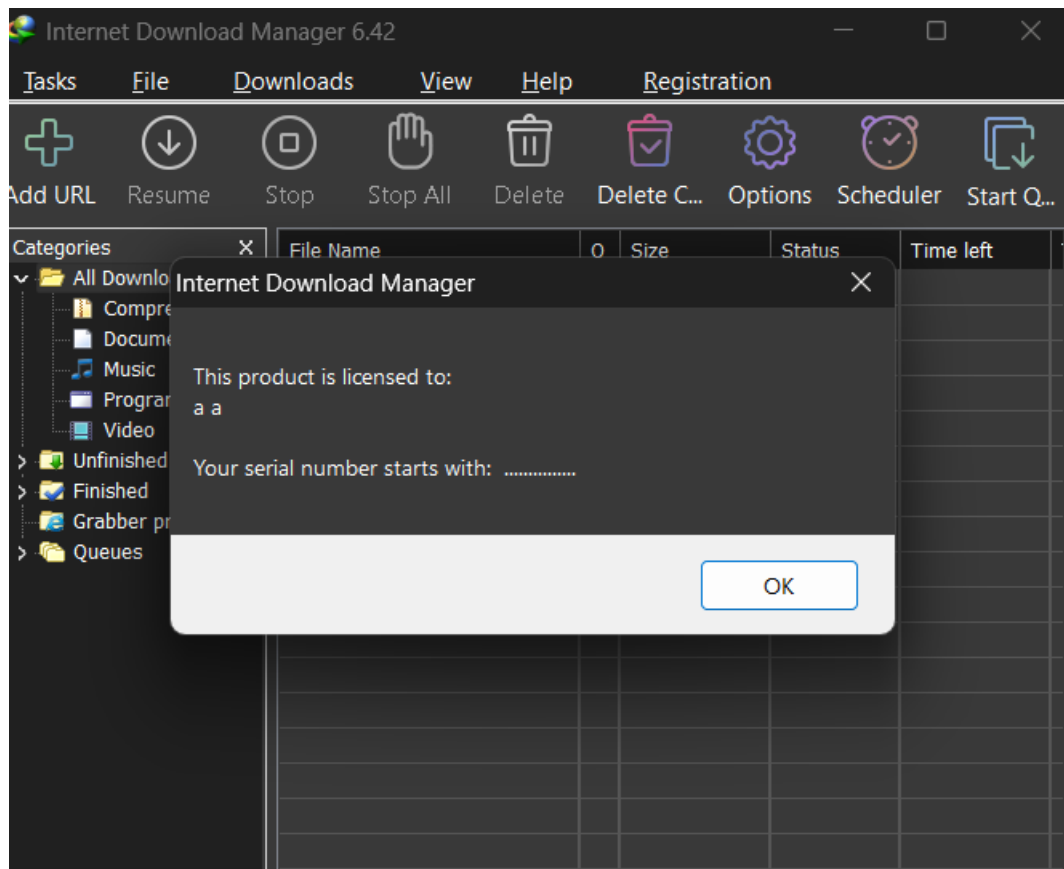
mã hiện Licensed).

00E8A2E1	BD 01000000	mov ebp,1	
00E8A2E6	391D DC8C1501	cmp dword ptr ds:[1158CDC],ebx	
00E8A2EC	0F84 91040000	je idman.E8A783	
00E8A2F2	8D4C24 18	lea ecx,dword ptr ss:[esp+18]	
00E8A2F6	E8 F581F7FF	call <idman.sub_E024F0>	
00E8A2FB	899C24 E4000000	mov dword ptr ss:[esp+E4],ebx	
00E8A302	8D4424 20	lea eax,dword ptr ss:[esp+20]	[esp+20]:Tp
00E8A306	50	push eax	
00E8A307	55	push ebp	
00E8A308	53	push ebx	

1.3.5. Kiểm tra kết quả thực nghiệm

Sau khi áp dụng bản vá và xuất tệp thực thi mới (idm6.exe), quá trình kiểm thử cho thấy:

- Phần mềm khởi động bình thường không có lỗi.
- Khi ấn vào Registration -> Registration, phần mềm hiển thị thông báo Licensed



2. Cách 2: Bypass bằng x32dbg để bỏ thông báo check Serial

2.1. Thu thập thông tin và cơ sở lý thuyết

Như đã trình bày, nhóm nghiên cứu xác định Internet Download Manager (IDMan) sử dụng cơ chế cấp phép kết hợp giữa xác thực ngoại tuyến (Offline Licensing) và kiểm tra trực tuyến (Online Validation). Khi người dùng nhập thông tin đăng ký, phần mềm sẽ lưu trữ các trường dữ liệu bao gồm Họ tên (FName, LName), Email và Số sê-ri (Serial Number) vào hệ thống Registry của Windows.

Tuy nhiên, do luồng kiểm tra này dựa trên các lệnh nhảy điều kiện (Conditional Jumps) trong mã máy để quyết định hướng đi của chương trình (hiện thông báo Licensed hoặc hiện thông báo Trial), nhóm đưa ra giả thuyết có thể can thiệp trực tiếp vào các lệnh này bằng kỹ thuật Dynamic Patching để ép chương trình xác nhận trạng thái bản quyền mà không cần một Serial hợp lệ từ máy chủ.

2.2. Truy vết quá trình phần mềm tìm kiếm hàm check serial

Để chứng minh giả thuyết trên, nhóm tiến hành sử dụng x32dbg, tìm kiếm chuỗi ký tự (Search for Strings) với từ khóa “incorrect serial”. Qua đó, nhóm định vị được điểm tham chiếu và đặt Breakpoint tại hàm thông báo sai serial number.

00F22882	E8 39E8FFFF	call <idman.sub_F213C0>	
00F22887	83C4 08	add esp,8	
00F2288A	85C0	test eax,eax	
00F2288C	74 04	je idman.F22892	
00F2288E	84DB	test b1,b1	
00F22890	74 12	je idman.F228A4	
00F22892	6A 00	push 0	
00F22894	68 D8DD0701	push idman.107DD08	107DD08:"Internet Download Manager"
00F22899	8BD0 F86E1601	mov ecx,dword ptr ds:[1166EF8]	ecx:PathAddBackslashw+E80, 01166EF8:&"You have entered incorrect Serial Number."
00F2289F	E9 6AFCFFFF	jmp idman.F2250E	
00F228A5	8B4D D0	mov ecx,dword ptr ss:[ebp-30]	ecx:PathAddBackslashw+E80
00F228A7	B8 838EA02F	mov eax,2FA0BE83	
00F228AC	F7E9	imul ecx	ecx:PathAddBackslashw+E80
00F228AE	C1FA 03	sar edx,3	
00F228B1	8BC2	mov eax,edx	
00F228B3	C1E8 1F	shr eax,1F	
00F228B6	03C2	add eax,edx	

00F22719	75 F9	jne idman.F22714	
00F2271B	2BC2	sub eax,edx	
00F2271D	75 16	jne idman.F22735	
00F2271F	6A 00	push 0	
00F22721	68 D8DD0701	push idman.107DD08	107DD08:"Internet Download Manager"
00F22723	A1 F86E1601	mov eax,dword ptr ds:[1166EF8]	01166EF8:&"You have entered incorrect Serial Number.\r\n\r\nPlease don't
00F22725	50	push eax	
00F2272C	8B4F 20	mov ecx,dword ptr ds:[edi+20]	ecx:NTUserPeekMessage+C
00F2272F	51	push ecx	ecx:NTUserPeekMessage+C
00F22730	E9 DEF0FFFF	jmp idman.F22513	
00F22735	8D85 A4010000	lea eax,dword ptr ss:[ebp+144]	[ebp+144]&"CloseHandleMainSwitch"

Mục tiêu của nhóm là khiến chương trình bỏ qua thông báo kiểm tra serial và hoạt động mà không cần xác nhận bản quyền online từ máy chủ.

Sau khi khởi động IDM và điền vào form đăng ký, nhóm có nhập sai Serial number với mục đích theo dõi xem chương trình gọi hàm nào để kiểm tra, trước đó nhóm đã đặt trước 1 break point ở địa chỉ F22639

00F22637	B3 20	mov bl,20	20: ' '
00F22639	8DA424 00000000	lea esp,dword ptr ss:[esp]	
00F22640	389D A4010000	cmp byte ptr ss:[ebp+1A4],bl	
00F22646	0F85 BF000000	jne idman.F2270B	
00F2264C	6A 32	push 32	
00F2264E	8D8D A5010000	lea ecx,dword ptr ss:[ebp+1A5]	
00F22654	51	push ecx	
00F22655	8D55 00	lea edx,dword ptr ss:[ebp]	
00F22658	52	push edx	edx:"com"
00F22659	E8 E2EEEEFF	call <idman.sub_E05540>	

Nhóm tiếp tục Debug, Step over (F8) qua từng bước để theo dõi thì nhóm có thấy ở dòng F2279C có lệnh nhảy đến địa chỉ F2271F

00F22781	8D85 A4010000	lea eax,dword ptr ss:[ebp+1A4]	
00F22787	8D50 01	lea edx,dword ptr ds:[eax+1]	
00F2278A	8D9B 00000000	lea ebx,dword ptr ds:[ebx]	
00F22790	8A08	mov cl,byte ptr ds:[eax]	
00F22792	40	inc eax	
00F22793	84C9	test cl,cl	
00F22795	75 F9	jne idman.F22790	
00F22797	2BC2	sub eax,edx	
00F22799	83F8 17	cmp eax,17	
00F2279C	75 81	jne idman.F2271F	
00F2279E	32DB	xor bl,bl	
00F227A0	B0 2D	mov al,2D	2D: '-'

00F22719	75 F9	jne idman.F22714	
00F2271B	2BC2	sub eax,edx	
00F2271D	75 16	jne idman.F22735	
00F2271F	6A 00	push 0	
00F22721	68 D8D0701	push idman.107DD08	107DD08:"Internet Download Manager"
00F22726	A1 F86E1601	mov eax,dword ptr ds:[1166EF8]	01166EF8:&"You have entered incorrect Serial Number.\r\n\r\nPlease don't
00F22728	50	push eax	
00F2272C	8B4F 20	mov ecx,dword ptr ds:[edi+20]	ecx:NTUserPeekMessage+C
00F2272F	51	push ecx	ecx:NTUserPeekMessage+C
00F22730	E9 DEFDFFFF	jmp idman.F22513	
00F22735	8D85 A4010000	lea eax,dword ptr ss:[ebp+1A4]	[ebp+1A4]&"C:\os\idman\idmanSwitch"

Ngay dưới F2271F là F22726 với hàm thông báo sai serial number, nên nhóm quyết định thay lệnh jne thành je ở dòng F2279C để chương trình không đi vào nhánh thông báo sai nữa

00F22792	40	inc eax	
00F22793	84C9	test cl,cl	
00F22795	75 F9	jne idman.F22790	
00F22797	2BC2	sub eax,edx	
00F22799	83F8 17	cmp eax,17	
00F2279C	74 81	je idman.F2271F	
00F2279E	32DB	xor bl,bl	
00F227A0	B0 2D	mov al,2D	2D: '-'
00F227A2	3885 A9010000	cmp byte ptr ss:[ebp+1A9],al	
00F227A8	75 10	jne idman.F227BA	
00F227AA	3885 AF010000	cmp byte ptr ss:[ebp+1AF],al	

Nhóm tiếp tục đi từng bước để theo dõi thì ở địa chỉ F2288C có lệnh je nhảy vào địa chỉ F22892, nếu tiếp tục thì chương trình sẽ rẽ vào thông báo sai serial nên nhóm quyết định chỉnh lệnh je thành jne ở cả 2 dòng F2288C và F22890

00F22881	50	push eax	
00F22882	E8 39EBFFFF	call <idman.sub_F213C0>	
00F22887	83C4 08	add esp,8	
00F2288A	85C0	test eax,eax	
00F2288C	74 04	je idman.F22892	
00F2288E	84DB	test bl,bl	
00F22890	74 12	je idman.F228A4	
00F22892	6A 00	push 0	
00F22894	68 D8DD0701	push idman.107DDD8	
00F22899	8B0D F86E1601	mov ecx,dword ptr ds:[1166EF8]	107DDD8:"Internet Download Manager"
00F2289F	E9 6AFCFFFF	jmp idman.F2250E	01166EF8:&"You have entered incorrect Serial Number.\r\n\r\nPlease don't mix 0(zero)
00F228A4	8B4D D0	mov ecx,dword ptr ss:[ebp-30]	
00F228A7	B8 83BEA02F	mov eax,2FA0BE83	
00F228AC	F7E9	imul ecx	
00F228AE	C1FA 03	sar edx,3	
00F228B1	8BC2	mov eax,edx	

00F22887	83C4 08	add esp,8	
00F2288A	85C0	test eax,eax	
00F2288C	75 04	jne idman.F22892	
00F2288E	84DB	test bl,bl	
00F22890	75 12	jne idman.F228A4	
00F22892	6A 00	push 0	
00F22894	68 D8DD0701	push idman.107DDD8	
00F22899	8B0D F86E1601	mov ecx,dword ptr ds:[1166EF8]	107DDD8:"Internet Download Manager"
00F2289F	E9 6AFCFFFF	jmp idman.F2250E	01166EF8:&"You have entered incorrect
00F228A4	8B4D D0	mov ecx,dword ptr ss:[ebp-30]	
00F228A7	B8 83BEA02F	mov eax,2FA0BE83	
00F228AC	F7E9	imul ecx	

Kết quả là chương trình đã thoát khỏi nhánh thông báo lỗi

00F2288A	85C0	test eax,eax	
00F2288C	75 04	jne idman.F22892	
00F2288E	84DB	test bl,bl	
00F22890	75 12	jne idman.F228A4	
00F22892	6A 00	push 0	
00F22894	68 D8DD0701	push idman.107DDD8	
00F22899	8B0D F86E1601	mov ecx,dword ptr ds:[1166EF8]	107DDD8:"Internet Download Manager"
00F2289F	E9 6AFCFFFF	jmp idman.F2250E	01166EF8:&"You have entered incorrec
00F228A4	8B4D D0	mov ecx,dword ptr ss:[ebp-30]	
00F228A7	B8 83BEA02F	mov eax,2FA0BE83	
00F228AC	F7E9	imul ecx	
00F228AE	C1FA 03	sar edx,3	
00F228B1	8BC2	mov eax,edx	
00F228B3	C1E8 1F	shr eax,1F	
00F228B6	03C2	add eax,edx	

Tiếp tục theo dõi thì nhóm có thấy 3 lệnh nhảy dài lên lại thông báo serial sai

00F22903	05D0	add edx,eax	
00F22907	8BC1	mov eax,ecx	
00F22909	2BC2	sub eax,edx	
00F2290B	75 04	jne idman.F22911	
00F2290D	85C9	test ecx,ecx	
00F2290F	75 02	jne idman.F22913	
00F22911	B3 01	mov bl,1	
00F22913	8B4D BC	mov ecx,dword ptr ss:[ebp-44]	
00F22916	B8 ED73484D	mov eax,4D4873ED	
00F22918	F7E9	imul ecx	
00F2291D	C1FA 04	sar edx,4	
00F22920	8BC2	mov eax,edx	
00F22922	C1E8 1F	shr eax,1F	
00F22925	03C2	add eax,edx	
00F22927	6BC0 35	imul eax,eax,35	
00F2292A	8BD1	mov edx,ecx	
00F2292C	2BD0	sub edx,eax	
00F2292E	0F85 EBFDFFFF	jne idman.F2271F	
00F22934	85C9	test ecx,ecx	
00F22936	0F84 E3FDFFFF	je idman.F2271F	
00F2293C	84DB	test bl,bl	
00F2293E	0F85 DBFDFFFF	jne idman.F2271F	
00F22944	6A 08	push 8	
00F22946	8D95 A4010000	lea edx,dword ptr ss:[ebp+1A4]	

Nhóm đã thử đảo ngược 3 lệnh để thoát khỏi nhưng không thành công nên nhóm quyết định nhờ hàm nhảy ở dòng F2290F để nhảy ra ngoài tới địa chỉ F22944 bằng cách thay đổi lệnh jne idman.F22913 thành jmp idman.F22946

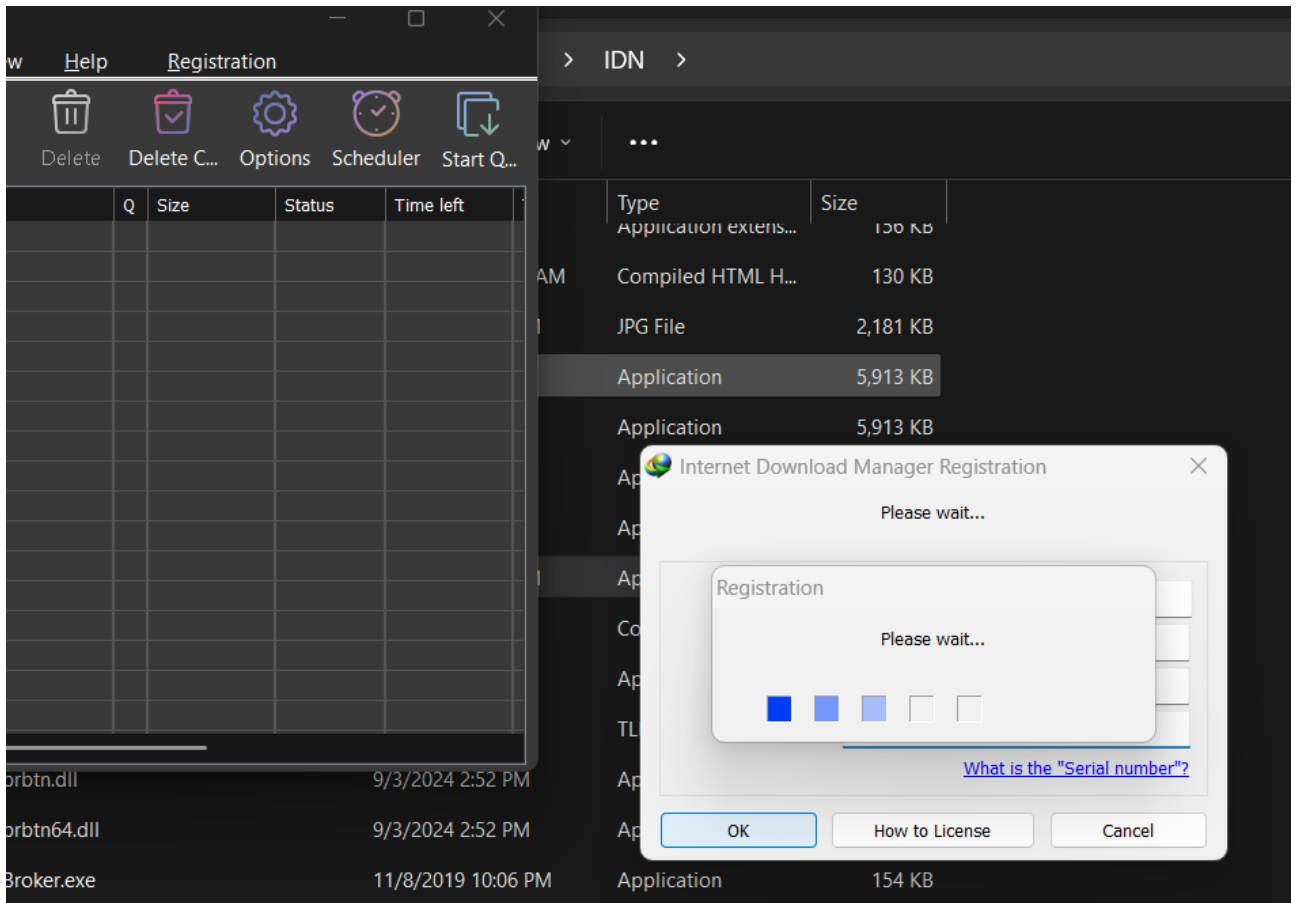
00F22927	6BC0 35	imul eax,eax,35
00F2292A	8BD1	mov edx,ecx
00F2292C	2BD0	sub edx,eax
00F2292E	0F84 EBFDFFFF	je idman.F2271F
00F22934	85C9	test ecx,ecx
00F22936	0F85 E3FDFFFF	jne idman.F2271F
00F2293C	84DB	test bl,bl
00F2293E	0F84 DBFDFFFF	je idman.F2271F
00F22944	6A 08	push 8
00F22946	8D95 A4010000	lea edx,dword ptr ss:[ebp+1A4]
00F2294C	52	push edx

00F22907	8BC1	mov eax,ecx
00F22909	2BC2	sub eax,edx
00F2290B	75 04	jne idman.F22911
00F2290D	85C9	test ecx,ecx
00F2290F	EB 33	jmp idman.F22944
00F22911	B3 01	mov bl,1
00F22913	8B4D BC	mov ecx,dword ptr ss:[ebp-44]
00F22916	B8 ED73484D	mov eax,4D4873ED
00F2291B	F7E9	imul ecx
00F2291D	C1FA 04	sar edx,4
00F22920	8BC2	mov eax,edx
00F22922	C1E8 1F	shr eax,1F
00F22925	03C2	add eax,edx
00F22927	6BC0 35	imul eax,eax,35
00F2292A	8BD1	mov edx,ecx
00F2292C	2BD0	sub edx,eax
00F2292E	0F85 EBFDFFFF	jne idman.F2271F
00F22934	85C9	test ecx,ecx
00F22936	0F84 E3FDFFFF	je idman.F2271F
00F2293C	84DB	test bl,bl
00F2293E	0F85 DBFDFFFF	jne idman.F2271F
00F22944	6A 08	push 8
00F22946	8D95 A4010000	lea edx,dword ptr ss:[ebp+1A4]

2.3. Kết quả cuối

Sau khi áp dụng bản vá và xuất tệp thực thi mới (idm5.exe), quá trình kiểm thử cho thấy:

Nhóm đã thành công trong việc ép chương trình chấp nhận Serial không hợp lệ thông qua kỹ thuật Patching lệnh nhảy điều kiện, dẫn đến việc xuất hiện hộp thoại "Please wait...". Tuy nhiên, hiện tượng phần mềm tự kết thúc sau vài giây cho thấy sự tồn tại của các cơ chế bảo vệ lớp thứ hai (Secondary Validation) hoặc lỗi thực thi do dữ liệu không đồng nhất.



File rarreg.key 1

CHƯƠNG III: KẾT QUẢ THỰC NGHIỆM

1. Tổng kết kết quả đạt được

Thông qua quá trình nghiên cứu và thực nghiệm, đề tài "Các kỹ thuật dịch ngược và ứng dụng vào Internet Download Manager" đã hoàn thành các mục tiêu đề ra ban đầu. Nhóm nghiên cứu đã hệ thống hóa được các cơ sở lý thuyết quan trọng của lĩnh vực Dịch ngược phần mềm (Reverse Engineering), bao gồm kỹ thuật phân tích tĩnh (Static Analysis), phân tích động (Dynamic Analysis), và kỹ thuật gỡ lỗi (Debugging) bằng các công cụ chuyên dụng như x32dbg.

Bằng cách can thiệp trực tiếp vào mã máy (Patching các lệnh rẽ nhánh je thành jne, jne thành je hoặc sử dụng lệnh nop), nhóm đã vô hiệu hóa thành công các lớp bảo vệ phân tầng (Multi-layered Protection), ép chương trình chuyển sang trạng thái đã đăng ký (Licensed) mà không cần số sê-ri hợp lệ. Thực nghiệm cũng cho thấy việc can thiệp vào các biến trạng thái trong Registry giúp làm rõ cách phần mềm lưu trữ và truy xuất dữ liệu cấp phép cục bộ.

2. Ý nghĩa thực tiễn

Kết quả của đề tài không nhằm mục đích khuyến khích hành vi vi phạm bản quyền, mà mang lại góc nhìn sâu sắc về tư duy bảo mật phần mềm. Nghiên cứu chỉ ra một điểm yếu phổ biến: Nếu phần mềm chỉ dựa vào các câu lệnh kiểm tra rẽ nhánh ở luồng thực thi cuối (Client-side validation) mà thiếu các cơ chế kiểm tra tính toàn vẹn (Integrity Check) mạnh mẽ hoặc kỹ thuật chống gỡ lỗi (Anti-Debugging), phần mềm sẽ dễ dàng bị đánh lừa bởi kỹ thuật Patching. Điều này giúp các nhà phát triển nhận ra tầm quan trọng của việc bảo vệ mã nguồn (Obfuscation) và kiểm tra chéo giữa các luồng xử lý để tăng cường tính an toàn.

3. Hạn chế và hướng phát triển

Dù đã đạt được kết quả nhất định, đề tài vẫn còn một số hạn chế. Việc can thiệp mã (Patching) ở mức độ cơ bản có thể khiến phần mềm gặp lỗi thực thi hoặc tự đóng (Crash) khi chạm phải các hàm kiểm tra phụ chạy ngầm hoặc khi phân

mềm thực hiện truy vấn xác thực từ máy chủ (Online Check). Trong tương lai, hướng phát triển của đề tài có thể mở rộng sang các nội dung phức tạp hơn như:

- Nghiên cứu sâu về Anti-Reversing: Phân tích các kỹ thuật phát hiện máy ảo (VM Detection) và các lệnh ngăn chặn Debugger mà IDM sử dụng để bảo vệ các hàm quan trọng.
- Kỹ thuật Unpacking: Tìm hiểu cách gỡ bỏ các lớp vỏ bọc bảo vệ (Packers) phức tạp như VMProtect hay Themida thường thấy trên các phiên bản phần mềm thương mại cao cấp.
- Giả lập phản hồi Server (Server Emulation): Nghiên cứu cách tạo ra các Local Server giả lập để đánh lừa hàm xác thực trực tuyến của phần mềm, giúp quá trình Bypass đạt được sự ổn định tuyệt đối mà không cần can thiệp quá sâu vào file thực thi.
- Đề xuất giải pháp bảo mật: Xây dựng mô hình xác thực đa nhân (Multi-factor validation) và cơ chế tự phục hồi mã lệnh dành cho nhà phát triển nhằm chống lại các công cụ dịch ngược hiện đại.

Tài liệu tham khảo:

- [1]<https://github.com/tpn/pdfs/blob/master/Introduction%20to%20x86%20Assembly.pdf> , “Introduction to x86 (32-bit) Assembly Language”, 03/04/2026.
- [2]<https://help.x64dbg.com/en/latest/introduction/index.html> , “x32dbg Documentation - The Open-source 32-bit Debugger for Windows”, 03/04/2026.
- [3]<https://beginners.re/> , “Reverse Engineering for Beginners - Understanding x86 Binary”, 03/04/2026.
- [4]<https://learn.microsoft.com/en-us/windows/win32/api/winreg/> , “Windows Registry API Reference (RegOpenKeyEx, RegQueryValueEx)”, 03/04/2026.
- [5]<https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html> , “Intel® IA-32 Architectures Software Developer’s Manual”, 03/04/2026.
- [6]<https://tuts4you.com/download.php?list.17> , “Collection of Reverse Engineering Tutorials for Beginners”, 03/04/2026.
- [7]<https://docs.microsoft.com/en-us/windows/win32/debug/system-error-codes> , “Windows System Error Codes and Debugging Symbols”, 03/04/2026.
- [8]<https://www.aldeid.com/wiki/X86-assembly/Instructions> , “X86 Assembly Instruction Set Reference (JMP, JE, JNE, NOP)”, 03/04/2026.

